

EPIC-B — Pseudonymous Handle and Profile System — Technical Spec

Status: Draft — for stakeholder review **Date:** 14 July 2026 **Author:** Adrian Hall (Technical Lead), drafted with Claude Code **Scope:** The second per-epic technical spec, building on the approved cross-cutting architecture in `docs/superpowers/specs/2026-07-14-askapeer-architecture-design.md` (“the architecture spec”) and directly continuing from `docs/superpowers/specs/2026-07-14-epic-a-verification-technical-spec.md` (“the EPIC-A spec”), which ends at the moment a member reaches `approved_verified` and is prompted to create a handle. Read both first, particularly the architecture spec’s Section 4.2 (community schema) and Section 4.4 (why identity/community are separated), and the EPIC-A spec’s Section 7 (authentication handoff).

Source of truth for product requirements: `docs/askapeer-prd-v0.1.md`, Section 9 (Anonymity & Safety Framework) governs this epic as much as Section 6.1’s feature table — EPIC-B is where the platform’s core anonymity guarantee first becomes a concrete data model and set of rules, not just a policy statement.

Contents

1. Scope
 2. Data model
 3. Handle creation
 4. Profile — what’s visible, what isn’t
 5. API endpoints
 6. Handle immutability
 7. Status effects (suspended/expelled)
 8. Follow handles (Should-have)
 9. Boundaries with other epics
 10. Failure modes and edge cases
 11. Non-functional notes specific to EPIC-B
 12. Test plan
 13. Open questions
-

1. Scope

In scope: handle creation immediately after `approved_verified` (the EPIC-A → EPIC-B handoff), handle-name validation rules, the public profile view (handle, kudos total, membership duration, post history), handle immutability

policy, and the effect of suspension/expulsion on a profile's visibility. Also specifies the Should-have "follow handles" feature (PRD Section 6.1) since it's a thin, purely-additive extension of the same `community_handles` table.

Out of scope for this spec (owned elsewhere): - Awarding/counting kudos itself — EPIC-D. This spec only owns the `kudos_total` *column* on the handle record; EPIC-D owns writing to it (Section 9). - Post/answer history *content* (the posts themselves) — EPIC-C. This spec only specifies that a profile surfaces a handle's post history, not the forum data model. - Moderation actions that *change* a handle's status — EPIC-F. This spec only specifies what a profile looks like once a status change has already happened (Section 7). - Notification preferences — EPIC-G. - The optional "professional contact link (e.g. LinkedIn URL)" the PRD's FD-5 discussion (Section 16) floats as a DM alternative — deliberately **not** built into this spec's data model. See Section 13; it isn't in the PRD's MoSCoW feature table (Section 6.1) as committed scope, and it appears to directly conflict with PRD Section 9.3's zero-tolerance rule, which names "deliberately identifying yourself" as an expulsion trigger. That conflict needs stakeholder resolution before any such field is built, not a unilateral design call here.

2. Data model

This epic owns `community_handles`, already defined in the architecture spec, Section 4.2, reproduced here with the one addition this spec requires:

```
community_handles
  id          uuid PK          -- the public
  "handle_id"
  member_id   uuid            -- FK exists for
referential integrity only;
                                -- readable solely
via IdentityService's role
                                -- (architecture
spec, Section 4.2)
  handle_name text unique      -- case-insensitively
unique; see Section 3
  kudos_total int default 0    -- written by EPIC-D,
read-only here
  member_since date            -- full date stored;
see Section 4 for why
                                -- display truncates
to year
  status      enum(active, suspended, expelled)
  created_at  timestamptz

community.handle_name_history      -- new: append-only,
admin/support use only
  id          uuid PK
  handle_id   uuid FK -> community_handles
  previous_name text
  changed_by  text              -- 'system' (first creation) or an
admin's member_id
  changed_at  timestamptz
```

Why `handle_name_history` **exists** even though Section 6 concludes handles shouldn't be freely self-service-changeable: the one legitimate case for a handle name changing at all is a moderator-forced rename (e.g. a handle name itself turns out to be identifying, or impersonates staff — see Section 3). When that happens, the *old* name must not simply vanish untraceably, since a moderator action changing a member-facing identifier is exactly the kind of thing the architecture spec's audit-logging philosophy (Section 4.4) says should leave a trail — otherwise a renamed handle's post history would look like it belonged to two different people with no record of why.

No new schema is introduced — this all lives in `community`, consistent with the architecture spec's principle that pseudonymous, peer-facing data never touches `identity`.

3. Handle creation

Continues directly from the EPIC-A spec, Section 7: the moment `identity.members verification_status` becomes `approved_verified`, the member is prompted to choose a handle. Until `community.handles` has a row for them, they cannot receive a full session token (EPIC-A spec, Section 7, point 3).

Validation rules (none of this is specified in the PRD directly — it's the technical detail the anonymity guarantee implies, and is flagged for review rather than assumed correct by construction):

- **Case-insensitive uniqueness** — `handle_name` collisions are checked ignoring case, so `DrSmith` and `drsmith` can't both exist (visually near-identical handles would otherwise be a natural phishing/impersonation vector in a trust-based community).
 - **No real-name plausibility check at creation time** — the platform cannot algorithmically know whether “Andrew_R_Physio” is someone's real name. This is handled by policy and reporting (PRD Section 9.3's “deliberately identifying yourself” rule), not creation-time validation, and is explicitly out of this spec's ability to enforce technically.
 - **Blocklist filtering**: profanity, slurs, and terms impersonating platform roles (`admin`, `moderator`, `askapeer`, `support`, and close variants) are rejected at creation time — this one *is* technically enforceable and prevents an obvious trust/impersonation failure mode before it reaches a human moderator.
 - **Length and character constraints**: a sensible bound (proposed: 3–30 characters, alphanumeric plus underscore/hyphen) — arbitrary but needs deciding; not a PRD requirement, flagged in Section 13.
 - **No reuse of a previously-expelled handle's exact name** — checked against `community.handle_name_history` as well as current `handles` rows, so a fresh registration can't casually re-adopt an identity an expelled member used, which could otherwise confuse other members about who they're now talking to.
-

4. Profile — what's visible, what isn't

Directly implements PRD Section 9.2's two lists. **Visible to peers:** `handle_name`, `kudos_total`, an approximate membership duration, and post/answer history (owned by EPIC-C, surfaced here by reference). **Never visible:** real name, employer/institution, geographic location, specialty, grade, or years of experience — none of which exist anywhere in the community schema in the first place (architecture spec, Section 4.2), so there's no field to accidentally leak; the guarantee is structural, not a display-layer filter over data that could be requested some other way.

`member_since` **display is year-only** (e.g. "Member since 2025"), even though the stored column is a full date. This isn't cosmetic: PRD Section 9.2's own example uses year granularity, and a precise join date is a plausible **correlation vector** — if a real person is known (from off-platform context) to have started a new job or finished registration on a specific date, an exact `member_since` timestamp narrows the search for which handle is theirs far more than a year does. Storing the full date (for internal/analytics use, e.g. KPI reporting in PRD Section 12) while only ever displaying the year is a deliberate storage/display split, not an oversight.

Post/answer history on a profile is exactly the posts EPIC-C already attributes to that `handle_id` — this spec doesn't duplicate that data, it specifies that the profile endpoint (Section 5) includes a reference to it.

5. API endpoints

Consistent with the architecture spec's Section 5.3 principles: versioned under `/v1`, and — critically for this epic — **no response shape from any of these endpoints ever includes** `member_id`, only `handle_id`.

```
POST /v1/handles
  auth: pending-scoped token, only valid once (EPIC-A spec,
  Section 7) and only
    callable when the caller's verification_status =
  approved_verified
  body: { handle_name }
  -> 201, { handle_id, handle_name }
  -> issues the full handle-scoped session JWT described in the
  architecture
    spec, Section 5.2, replacing the pending-scoped token
  -> 409 if handle_name fails uniqueness/blocklist validation
  (Section 3)

GET /v1/handles/:handle_id
  -> public profile: { handle_name, kudos_total, member_since
  (year only),
    status, post_history_ref }
  -> if status = expelled: handle_name and member_since still
  shown (removing
    them would create confusing gaps in threads where the
  expelled handle
    posted historically), kudos_total still shown, but no ability
  to interact
```

(follow, kudos) – see Section 7

GET /v1/handles/me

auth: handle-scoped token

-> same shape as above, plus fields only the member themselves needs

(e.g. notification-preference links – owned by EPIC-G, referenced here)

--- admin-only, moderator-role JWT claim required ---

POST /v1/admin/handles/:handle_id/rename

body: { new_handle_name, reason }

-> writes community.handle_name_history, updates community.handles.handle_name

-> used only for the moderator-forced-rename case (Section 2); not a

self-service endpoint

6. Handle immutability

Members cannot self-service rename their handle after creation. This isn't explicitly stated in the PRD, but follows from the anonymity model's own logic: kudos, post history, and "member since" are all reputation signals *of the handle*, and a freely renameable handle would let a member launder a damaged reputation (e.g. after public disagreement or a formal warning short of expulsion) by simply picking a new name while keeping the same underlying member_id — undermining the "ideas win on merit" thesis the PRD's introduction states as the platform's core value, since merit is tracked per-handle. It also removes a legitimate concern the architecture spec doesn't otherwise address: if renaming were self-service, a rename would need its own audit trail for exactly the reasons Section 2's handle_name_history table exists, but for a much higher volume of events.

The one exception (Section 5) is a moderator-forced rename, when the handle name itself becomes a problem (e.g. later found to be identifying, or an impersonation attempt that slipped past the Section 3 blocklist). This is a moderation action in substance and probably belongs in the moderation queue's action types (EPIC-F) rather than being a bespoke EPIC-B-only workflow — flagged in Section 13 as a boundary to confirm with the EPIC-F spec once it's written.

7. Status effects (suspended/expelled)

Reuses the community.handles.status enum already defined in the architecture spec (Section 4.2): active, suspended, expelled. This epic only specifies profile-visibility consequences; the *decision* to change status is EPIC-F's.

Status	Profile visibility	Interaction
active		Followable, kudos-able

Status	Profile visibility	Interaction
	Full profile as Section 4	
suspended	Full profile, unchanged	Not followable/kudosable while suspended (temporary)
expelled	Full profile, unchanged (see Section 5's GET / v1/handles/:handle_id note)	Not followable/kudosable (permanent)

Why an expelled handle's profile and history stay visible rather than being deleted or hidden: PRD Section 9.3's zero-tolerance rule is about *expelling the member*, not erasing the community's record of what they contributed — removing their post history would break threads other members' replies depend on, and the PRD's own right-to-erasure design (architecture spec, Section 9) already establishes the precedent of retaining de-linked community content rather than deleting it. The handle itself becomes inert (can't log in, can't be interacted with going forward), but it doesn't vanish from the archive.

8. Follow handles (Should-have)

PRD Section 6.1 lists this as a Should-have, not Must-have, but it's specified here since it's a thin addition to this epic's own table, not a separate module:

```
community.handle_follows
  follower_handle_id  uuid FK -> community.handles
  followed_handle_id  uuid FK -> community.handles
  created_at          timestampz
  primary key (follower_handle_id, followed_handle_id)
```

```
POST    /v1/handles/:handle_id/follow    auth: handle-scoped
token
DELETE  /v1/handles/:handle_id/follow    auth: handle-scoped
token
GET      /v1/handles/me/following          -> list of followed
handle_ids
```

Consistent with the PRD's framing ("You follow their ideas, not their identity") — following is purely a `handle_id`-to-`handle_id` relationship; nothing here differs from any other community-schema interaction in terms of the identity boundary. The personalised-feed use of this table (PRD Section 6.1's other Should-have, "Personalised feed") is EPIC-C's concern, not this spec's; this section only owns the follow relationship's existence and storage.

Self-follow (`follower_handle_id` = `followed_handle_id`) is rejected at the API layer — not a meaningful action, and cheap to exclude.

9. Boundaries with other epics

Several fields and behaviors on `community.handles` are written by other epics; this section exists so those specs don't have to re-derive the boundary:

- **kudos_total**: EPIC-D's write path (an award increments it). EPIC-B defines the column and reads it for profile display; EPIC-D is the only writer. This mirrors the architecture spec's general pattern of one clear writer per shared piece of state.
 - **status**: EPIC-F's write path (moderation decisions). EPIC-B defines the enum and its profile-visibility consequences (Section 7); EPIC-F is the only writer.
 - **Post/answer history**: entirely EPIC-C's data (`community.posts/community.comments` keyed by `handle_id`); EPIC-B's profile endpoint only references it.
 - **Notification preferences surfaced on** `/v1/handles/me`: EPIC-G's data; included in that endpoint's response for convenience, not owned here.
-

10. Failure modes and edge cases

- **Two applicants race to claim the same handle name**: the database's case-insensitive unique constraint (Section 2/3) is the actual source of truth — the API returning 409 on a race is expected and the client should offer alternatives, not retry blindly.
 - **A moderator-forced rename collides with an existing handle**: the rename endpoint (Section 5) runs the same blocklist/uniqueness validation as creation; a rename that would collide is rejected, requiring the admin to pick a different replacement name.
 - **An expelled member re-registers under EPIC-A with the same professional registration**: EPIC-A's uniqueness constraint (its spec, Section 2) only blocks non-rejected prior members, and expulsion doesn't change `identity.members.verification_status` to rejected — this is a gap between the two specs worth resolving explicitly (Section 13), since as written a permanently expelled practitioner could plausibly re-verify and create a brand-new handle, which seems to contradict "immediate permanent expulsion."
 - **Handle deleted mid-session** (e.g. moderator expels while the member is actively posting): the existing access token remains valid for up to its ~15-minute lifetime (same mechanic the architecture spec, Section 7.2, already describes for suspension/expulsion generally); no EPIC-B-specific handling needed beyond what that section already establishes.
-

11. Non-functional notes specific to EPIC-B

- **No new identity-schema access**: this epic's endpoints operate entirely under a `community-schema` database role; the cross-schema regression test the architecture spec requires (Section 9) should be extended to cover this epic's endpoints specifically once built, the same way the EPIC-A spec's test plan does for its own.
- **Handle-name blocklist maintenance**: the profanity/impersonation blocklist (Section 3) needs an update mechanism (config-driven, not a code change per new term) — small detail, but worth deciding where

that list lives (a config table vs. an application constant) before build; not specified further here since it has no architectural weight either way.

12. Test plan

- **Uniqueness:** case-insensitive collision is rejected; visually-confusable-but-technically-distinct names (e.g. `drsmith` vs `dr_smith`) are *not* blocked by this rule — confirms the constraint is doing exactly what Section 3 describes, no more.
 - **Blocklist:** creation attempts using reserved terms (`admin`, `moderator`, `askapeer`, close variants) are rejected; legitimate handles containing a blocklisted term as a substring only (e.g. a hypothetical clinical term that happens to contain a blocked fragment) are not — needs a real blocklist implementation decision, not just a naive substring match, to avoid false positives.
 - **Expelled-name reuse:** registering a handle name matching an entry in `handle_name_history` is rejected even though the *current* `handles` table has no row with that name.
 - **Profile field boundary:** an automated test asserting the `GET /v1/handles/:handle_id` response DTO has no path to `member_id`, `legal_name`, or any other identity-schema field — mirrors the EPIC-A spec's Section 10 approach.
 - `member_since` **display:** confirm the API/display layer truncates to year even though the stored value is a full date; a regression test on this specifically, since it's a privacy-relevant behavior that a well-intentioned future refactor could quietly "fix" by exposing the full date.
 - **Status-gated interactions:** a suspended or expelled handle cannot be followed or kudos'd (once EPIC-D exists to test against), but its existing profile and history remain readable.
-

13. Open questions

- **The FD-5 professional-contact-link tension** (Section 1): PRD Section 16 (FD-5) recommends letting members add a LinkedIn URL to their profile as a DM alternative, but PRD Section 9.3 lists "deliberately identifying yourself" as an immediate-expulsion offence. These two sections of the same document appear to be in direct tension. Needs Paul Gouge/Andrew Renshaw's attention before FD-5 is closed — flagging here since EPIC-B's data model is the place this would land if resolved in favour of building it, and right now it deliberately isn't built.
- **Handle length/character rules** (Section 3): the 3–30 character, alphanumeric-plus-underscore/hyphen proposal is this spec's own placeholder, not a PRD requirement — needs a real decision, though it has no architectural consequence either way.
- **Moderator-forced rename as a first-class moderation action:** should `POST /v1/admin/handles/:handle_id/rename` (Section 5) actually live in EPIC-F's action-type enum (`remove_content`, `warn`, `suspend`, `expel` per the architecture spec, Section 7.2) rather than as a separate EPIC-B-only endpoint? Flagged for reconciliation once the EPIC-F spec is written.
- **Expelled-member re-registration gap** (Section 10): EPIC-A's uniqueness constraint doesn't currently prevent a genuinely expelled practitioner from re-verifying under the same registration and creating a new handle, since expulsion doesn't set `verification_status` =

rejected. Needs a cross-epic decision: either expulsion should also flip the `identity.members` status, or EPIC-A's registration check needs to consult moderation history, not just verification history. Worth resolving before EPIC-F is written, since EPIC-F is what actually performs expulsions.

- **Blocklist storage/maintenance mechanism:** config table vs. code constant — an implementation detail with no architectural weight, but needs picking before build.